Assignment #3

1. Analyze and compare the performance of two all-pairs shortest path algorithms: Johnson's Algorithm and Floyd–Warshall Algorithm.

Date of Performance: 29/07/26

Date of Submission: 19/08/26

Student ID: 00724105101122

Name: Rohan Rahim

Group: C1

Objective:

Analyze and compare the performance of two all-pairs shortest path algorithms:
- Johnson's Algorithm
- Floyd–Warshall Algorithm

Evaluate them on two graphs with 50 nodes each:
1. Sparse Graph: 45 nodes, 100–150 randomly generated edges.
2. Dense Graph: 45 nodes, complete graph (1225 edges).

Tasks:

1. Graph Generation
    a. Generate a sparse weighted directed graph (45 nodes, 150-200 edges).
    b. Generate a complete weighted directed graph (45 nodes, 990 edges).
    c. Assign random weights.

2. Algorithm Implementation
    a. Johnson's Algorithm
    b. Floyd Warshall Algorithm

3. For each run, measure:
    - Execution time (s)
    - Estimated energy consumption (J)
      Energy (J) = CPU's approximate average power x Execution Time (s)
    - Peak memory usage (KB)
    - Estimated CO2 emissions (Bangladesh) Energy (kWh) = Energy (J) / 3,600,000
      CO2 (kg) = Energy (kWh) x 0.62

Experimental Results:

Algorithm	Graph Type	Nodes	Edges	Execution Time (s)	Energy (J)	Peak Memory (KB)	CO_2 Emissions (kg, BD)
Johnson	Sparse	45	170	0.000998815	0.064923	6640	0.00000001
Floyd–Warshall		45	170	0.000531864	0.0345712	6640	0.00000001
Johnson	Dense	45	990	0.00230478	0.149811	6640	0.00000003
Floyd–Warshall		45	990	0.000672942	0.0437412	6640	0.00000001

Discussion:

- **Dense Graphs (E = 990):** Floyd–Warshall is faster and uses almost half the energy of Johnson's algorithm. 2D array lookups benefit from CPU cache locality, whereas Johnson's algorithm suffers from min-heap priority queue management and overheads.

- **Sparse Graphs (E = 170):** While Johnson's algorithm has a superior theoretical time complexity $O(V^2 \log V + VE)$ vs $O(V^3)$ Floyd–Warshall remains faster at this scale (V = 45). At small vertex counts, constant-factor overheads from STL priority queue allocations outweigh loop iteration savings.

Overall Summary:

For small graph sizes (V <= 50), Floyd–Warshall is the practical choice regardless of density due to lower constant-factor overhead and superior CPU cache performance. Johnson's Algorithm becomes the preferred choice when scaling to large, sparse graphs where matrix iterations become computationally prohibitive.

Code:

```
#include<bits/stdc++.h>
using namespace std;

// OS-Specific Headers for Memory Tracking
#if defined(_WIN32)
  #include <windows.h>
  #include <psapi.h>
#elif defined(__APPLE__)
  #include <mach/mach.h>
#elif defined(__linux__)
  #include <sys/resource.h>
  #include <fstream>
  #include <unistd.h>
#endif

// Constants
const double CPU_POWER_WATTS = 65.0;
const double BD_EMISSION_FACTOR = 0.62; // kg CO₂ / kWh
const int INF = 1e9;

// Cross-platform function to get Peak RSS (Resident Set Size) in KB
size_t getPeakMemoryKB() {
#if defined(_WIN32)
  PROCESS_MEMORY_COUNTERS pmc;
  if (GetProcessMemoryInfo(GetCurrentProcess(), &pmc, sizeof(pmc))) {
    return pmc.PeakWorkingSetSize / 1024;
  }
#elif defined(__APPLE__)
  struct mach_task_basic_info info;
  mach_msg_type_number_t count = MACH_TASK_BASIC_INFO_COUNT;
  if (task_info(mach_task_self(), MACH_TASK_BASIC_INFO, (task_info_t)&info, &count) == KERN_SUCCESS) {
    return info.resident_size_max / 1024; // macOS returns max bytes
  }
#elif defined(__linux__)
  struct rusage usage;
  if (getrusage(RUSAGE_SELF, &usage) == 0) {
    return usage.ru_maxrss; // Linux returns maxrss in KB directly
  }
#endif
  return 0;
}

// Adjacency List Edge Structure for Johnson's Algorithm
struct Edge {
  int to;
  int weight;
};

// Bellman-Ford (Computes h-values for reweighting)
bool bellmanFord(int V, const vector<vector<Edge>>& adj, vector<int>& h) {
  h.assign(V + 1, INF);
  h[V] = 0; // Virtual node V connected to all vertices with 0 weight

  vector<vector<Edge>> extAdj = adj;
  extAdj.push_back(vector<Edge>());
  for (int i = 0; i < V; ++i) {
    extAdj[V].push_back({i, 0});
  }

  int totalV = V + 1;
  for (int iter = 0; iter < totalV - 1; iter++) {
    bool updated = false;
    for (int u = 0; u < totalV; ++u) {
      if (h[u] == INF) continue;
      for (const auto& edge : extAdj[u]) {
        if (h[u] + edge.weight < h[edge.to]) {
          h[edge.to] = h[u] + edge.weight;
```

```
                updated = true;
            }
        }
    }
    if (!updated) break;
  }

  // Negative cycle detection
  for (int u = 0; u < totalV; ++u) {
    if (h[u] == INF) continue;
    for (const auto& edge : extAdj[u]) {
      if (h[u] + edge.weight < h[edge.to]) {
        return false;
      }
    }
  }
  return true;
}

// Dijkstra (Runs on non-negative reweighted graph)
void dijkstra(int src, int V, const
vector<vector<Edge>>& reweightedAdj, vector<int>&
dist) {
  dist.assign(V, INF);
  dist[src] = 0;

  priority_queue<pair<int, int>, vector<pair<int, int>>,
greater<pair<int, int>>> pq;
  pq.push({0, src});

  while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();

    if (d > dist[u]) continue;

    for (const auto& edge : reweightedAdj[u]) {
      if (dist[u] + edge.weight < dist[edge.to]) {
        dist[edge.to] = dist[u] + edge.weight;
        pq.push({dist[edge.to], edge.to});
      }
    }
  }
}

// Algorithm 1: Johnson's Algorithm
vector<vector<int>> johnson(int V, const
vector<vector<Edge>>& adj) {
  vector<int> h;
  if (!bellmanFord(V, adj, h)) {
    cerr << "Error: Graph contains a negative-weight
cycle.\n";
    return {};
  }

  // Reweight edges: w'(u, v) = w(u, v) + h[u] - h[v]
  vector<vector<Edge>> reweightedAdj(V);
  for (int u = 0; u < V; ++u) {
    for (const auto& edge : adj[u]) {
      reweightedAdj[u].push_back({edge.to,
edge.weight + h[u] - h[edge.to]});
    }
  }

  // Run Dijkstra for all vertices
  vector<vector<int>> allDist(V, vector<int>(V, INF));
  for (int u = 0; u < V; ++u) {
    vector<int> distFromU;
    dijkstra(u, V, reweightedAdj, distFromU);
    for (int v = 0; v < V; ++v) {
      if (distFromU[v] != INF) {
        allDist[u][v] = distFromU[v] - h[u] + h[v];
      }
    }
  }
  return allDist;
}

// Algorithm 2: Floyd-Warshall Algorithm
vector<vector<int>> floydWarshall(int V, const
vector<vector<int>>& distMatrix) {
  vector<vector<int>> dist = distMatrix;

  for (int k = 0; k < V; ++k) {
    for (int i = 0; i < V; ++i) {
      for (int j = 0; j < V; ++j) {
        if (dist[i][k] != INF && dist[k][j] != INF) {
          dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
        }
      }
    }
  }
  return dist;
}

int main() {
  constexpr int V = 45;

  /// OPTION 1: FIXED SPARSE DIRECTED GRAPH
  // 45 Vertices, ~170 Directed Edges (i < v), Negative
Weights [-10, 39]
  // vector<vector<Edge>> adjList(V);
  // vector<vector<int>> adjMatrix(V, vector<int>(V,
INF));
  // for (int i = 0; i < V; ++i) adjMatrix[i][i] = 0;

  // for (int i = 0; i < V; ++i) {
  //     for (int k = 1; k <= 4; ++k) {
  //         int v = i + k;
  //         if (v < V) { // Strictly forward edges (i < v) ->
Guarantees no negative cycles
  //             int weight = ((i * 7 + k * 13) % 50) - 10; //
Weights in range [-10, 39]
  //             adjList[i].push_back({v, weight});
  //             adjMatrix[i][v] = weight;
  //         }
  //     }
  // }

  // OPTION 2: FIXED DENSE DIRECTED GRAPH
  // 45 Vertices Directed Complete Graph (990 Edges),
Negative Weights [-15, 44]
  cout << "=== Running on Fixed Dense Directed
Graph (V = 45, E = 990) ===\n";
  vector<vector<Edge>> adjList(V);
  vector<vector<int>> adjMatrix(V, vector<int>(V, INF));
  for (int i = 0; i < V; ++i) adjMatrix[i][i] = 0;

  // Directed complete graph: 45 * 44 / 2 = 990
directed forward edges (i < j)
  for (int i = 0; i < V; ++i) {
    for (int j = i + 1; j < V; ++j) {
      int weight = ((i * 11 + j * 17) % 60) - 15; // Weights in
range [-15, 44]
      adjList[i].push_back({j, weight});
      adjMatrix[i][j] = weight;
    }
  }
```

```
    // High-resolution cross-platform timer
    auto start = chrono::high_resolution_clock::now();

    // -------- Your algorithm here --------

    johnson(V, adjList);
    // floydWarshall(V, adjMatrix);

    // -------------------------------------
    auto end = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = end - start;

    // Calculations
    double elapsed_seconds = elapsed.count();
    double energy = CPU_POWER_WATTS *
elapsed_seconds; // Joules
    double energy_kWh = energy / 3.6e6;              // kWh
    double co2 = energy_kWh * BD_EMISSION_FACTOR;
// kg CO2

    // Print Results
    cout << "Execution time: " << elapsed_seconds << "
seconds\n";
    cout << "Estimated energy consumption: " <<
energy << " Joules\n";
    size_t peakMem = getPeakMemoryKB();
    if (peakMem > 0) {
        cout << "Peak Memory Usage: " << peakMem << "
KB\n";
    } else {
        cout << "Failed to get memory info.\n";
    }
    printf("Estimated CO2 emissions (Bangladesh): %.8f
kg\n", co2);

    return 0;
}
```